Microsoft

PR Regression Safety

Why green CI is not “safe to merge” — and the phased gate that fixes it

Research findings · microsoft/physical-ai-toolchain · June 2026

Green CI has missed every costly regression

8 vs 0

Eight runtime regressions and test-integrity gaps reconstructed from this repo's own history — zero caught by the green, CPU-only pipeline.

- Most are runtime, GPU, or interpreter specific — what CPU CI never exercises
- Two were caused by CI itself: path-filter bugs switched tests off for weeks
- In this project a green check has repeatedly meant nothing was actually tested

The eight, in one place

Five named failures carry the argument; the full catalogue is in the appendix

#809 — interpreter ABI	RL lock resolved for Python 3.12 against a 3.11 runtime → four cascading ABI failures
#790 — LeRobot interpreter	LeRobot needs Python ≥ 3.12 against an OSMO 3.11 runtime
#958 — torch / CUDA	a torch 2.10→2.11 security bump pulled CUDA 13 bindings → libcudart break; live desync today
#691 / #547 — tests off	path-filter bugs silently disabled fuzz, data-pipeline, and training tests for weeks
churn	starlette churned 0.52→1.0→1.3 across ~7 dependency PRs and a downgrade-then-rebump in ~12 days

What this asks for

- Phase 0 — risk-aware Dependabot grouping · config only · ~$0
- Phase 1 — GPU-free smoke gate on every PR · standard runners · ~$0
- Phase 2 — safe automation: auto-merge + review · standard runners · ~$0
- Phase 3 — gated GPU end-to-end (the capstone) · funded GPU compute
- Evidence: repo history, configs, PRs, and an executed prototype — cited in the appendix

The stack under test — why this repo is hard

CI/CD generalists meet the robotics runtime: these concepts drive every later recommendation

Isaac Lab	GPU simulator (needs Vulkan); ships its own Python 3.11 runtime
CUDA / driver ABI	torch and CUDA wheels are compiled contracts — they must match the GPU driver
MIG	one datacentre GPU sliced into isolated instances; a config, not a library
AzureML / OSMO	submit-and-poll: CI sends a job spec, a scale-from-zero GPU pool runs it
uv / uv.lock	fast resolver plus an exact pinned graph, one per subproject

Dependency intake today

21 ecosystem blocks (9 uv · 3 npm · 4 Terraform · 3 Docker · 1 Go · 1 Actions), every group a wildcard

.github/dependabot.yml

```
updates:
- package-ecosystem: uv
  directory: /training/rl
  groups:
    training-dependencies:
      patterns: ["*"]          # catch-all: no risk split
  ignore:
  - dependency-name: marshmallow
    versions: [">=4.0.0"]      # reactive pin, post-breakage
  schedule: { interval: weekly, day: monday }
# ... repeated x21: npm, 10x uv, terraform, go, docker, actions
```

21 contexts = several independent runtimes

Not one app with one lock — the blast radius of a bump is wildly uneven

Training · RL	Isaac Lab SKRL/RSL-RL (Py 3.11, numpy 1.26); AzureML + OSMO GPU jobs
Training · IL	LeRobot ACT/Diffusion; full AzureML command-job + pipeline
Evaluation · SIL	ONNX / Torch inference and the eval container image
Dataviewer	FastAPI backend + React 19 frontend + 2 Docker images
Infrastructure	4 Terraform roots (AKS, DNS, VPN, automation) + Go contract tests

CI and automation today

All test CI is CPU-only; the torch pin desyncs from the lock; one advisory agent, read-only

```
.github/workflows/pytest-training.yml

jobs:
  test:
    runs-on: ubuntu-latest        # 4 vCPU - 16 GB - ~14 GB disk - NO GPU
    steps:
      - uses: actions/setup-python@...   # python-version: '3.12'
      - run: uv sync --group dev
      - run: uv pip install torch==2.11.0      # force-installed
# training/il/lerobot/uv.lock pins torch 2.10.0  ->  live desync (#958)
```

The failure map

Each incident against the phase that catches it — the spine of the plan

	Today	P0 group	P1 smoke	P2 auto	P3 GPU
#809 interpreter/ABI Py3.12 lock vs 3.11 runtime	✗	–	✓	–	–
#790 LeRobot Py≥3.12 vs OSMO 3.11 runtime	✗	–	~	–	–
#958 torch / CUDA resolution vs device ABI	✗	~	~	–	✓
#691/#547 tests off path-filter silently skipped	✗	–	✓	–	–
churn · starlette ×7 noise / wasted reviews	✗	✓	–	✓	–

✓ caught/prevented ~ partial — reduces risk – n/a ✗ missed today. #958 splits: ~ resolution caught early; ✓ the device-ABI break needs Phase 3.

Phase 0

Dependency intake

From blind wildcard intake to risk-aware grouping — config only, ~$0

Intake has no risk signal

- 21 ecosystem blocks, every one a wildcard catch-all — no split by risk
- A harmless patch and a CUDA-breaking major are batched identically
- ~7 ignore-pins added only after a breakage (torch, numpy, marshmallow…)
- No stability window: a bump can open the day it is published

What others do — group by risk, add a cooldown

Real, copyable config: vercel/ai splits prod vs dev; huggingface/transformers adds a window

vercel/ai · huggingface/transformers

```
# vercel/ai - split production vs development (npm)
groups:
  packages-production:
    dependency-type: production
    update-types: [minor, patch]
  packages-development:
    dependency-type: development
    update-types: [minor, patch]
# huggingface/transformers - 7-day stability window
cooldown:
  default-days: 7
```

Recommendation — split by risk, keep security fast

Batch the safe, isolate majors, add a window — and never batch security

Today — wildcard catch-all

```
groups:
  training-deps:
    patterns: ["*"]
ignore:
- dependency-name:
    marshmallow
  versions: [">=4.0.0"]
# x21 blocks
# patch == major
```

Proposed — risk split + cooldown

```
groups:
  prod-min-patch:
    dependency-type:
      production
    update-types:
      [minor, patch]
# majors isolated
cooldown:
  default-days: 7
# security: ungrouped
```

Renovate — a scoped spike, not a switch

One real residual gap; weigh it on evidence (full adoption data in the appendix)

The gap	Dependabot groups are per-ecosystem — still 4+ PRs/cycle; no one-config view
Not the gap	uv support — Dependabot has it since 2025 and this repo already uses it
The cost	the Mend App needs org approval; renovatebot/github-action avoids it entirely
The decision	time-boxed spike via the Action; switch only if PR volume drops without losing the security lane

Phase 1

GPU-free smoke gate

Catch the resolution, import, and interpreter breaks on every PR — ~$0

A green check tests nothing real

- All test CI is CPU-only ubuntu-latest — no GPU, no Isaac, no real runtime image
- The interpreter and ABI breaks fail on the real runtime, which CI never loads
- Path-filter bugs (#691, #547) switched tests off for weeks — still green
- So the gate must run inside the real image, not a generic runner

Two depths: Tier 0 and Tier 1

Tier 0 — venv, seconds, every PR

- `uv lock --check` — resolution drift *(today)*
- YAML schema-validate, `shellcheck` *(today)*
- import in a CPU venv + `--help` *(add)*
- Mostly already on — one probe to add
- Caveat: CPU wheels ≠ the production CUDA graph

Tier 1 — inside the real image, minutes

- `docker run` the ACTUAL runtime image *(add)*
- reinstall the PR's lock as prod does
- import on the real interpreter (Isaac = 3.11)
- Catches #809, probably #790 — deterministically
- Path-gated; bounded by disk, not capability

Tier 1 — import inside the real image

The recipe that catches the #809 class — no GPU; it fails at import

Proposed: isaac-import-smoke.sh

```
# Pull the REAL runtime image and reinstall the PR's lock the way prod does,
# then run the launcher's import-check mode (same graph as training, stops
# before AppLauncher — the only GPU step).
docker run --rm --platform linux/amd64 \
  nvcr.io/nvidia/isaac-lab:2.3.2 bash -lc '
    /isaac-sim/kit/python/bin/python3 -V        # 3.11 — the real runtime
    uv export --frozen --no-emit-project --directory training/rl \
      | uv pip install --system --no-deps -r -  # <- the #809 3.12-lock fails HERE
    python -m training.rl.scripts.launch --mode import-check  # ABI net: graph loads?
'   # all on CPU — no GPU touched
```

We ran it — it catches #809 on CPU

Executed this session inside the actual Isaac image, CPU only

```
prototype result (this session)

# EXECUTED inside nvcr.io/nvidia/isaac-lab:2.3.2 (amd64 emulated), CPU only:
$ docker run ... --entrypoint .../python3 isaac-lab:2.3.2 -c 'print(sys.version)'
  3.11.13                          # the real Isaac runtime

$ pip install .   # a dep with requires-python = ">=3.12"  (the #809 break)
  ERROR: Package requires a different Python: 3.11.13 not in '>=3.12'

# and against the repo's REAL training/rl lock, on the host:
$ uv lock --check --python 3.12
  error: ... incompatible with the project's requires-python `==3.11.*`
```

Recommendation — Phase 1a + 1b

1a Tier 0 every PR · 1b Tier 1 real-image, path-gated · one fail-safe required check

Proposed: .github/workflows/smoke-cpu.yml

```yaml
jobs:
  smoke:
    runs-on: ubuntu-latest
    steps:
      - run: uv lock --check                          # resolution drift
      - run: uv pip install torch --index-url https://download.pytorch.org/whl/cpu
      - run: python -c "import lerobot, torch"      # ABI / import smoke
      - run: ./training/rl/scripts/submit-osmo-training.sh --config-preview
      - uses: jlumbroso/free-disk-space@...
      - run: docker build -f evaluation/sil/docker/Dockerfile.lerobot-eval .
```

Phase 2

Safe automation

Reclaim reviewer time without lowering the bar — ~$0

Every low-risk bump waits on a human

- Dependabot cannot merge its own PRs — trivial patches queue for a click
- High-risk and low-risk updates get identical, manual handling
- ~24 PRs/week of reviewer toil, most of it trivial

Recommendation — auto-merge, scoped tight

- - `auto` merges only AFTER the smoke gate is green — patch-only, dev/docs/actions, no runtime/GPU, no security

Proposed: auto-merge workflow

```
on: pull_request_target          # base context; do NOT checkout the PR head here
permissions: { contents: write, pull-requests: write }   # NOT id-token
jobs:
  automerge:
    steps:
      - uses: dependabot/fetch-metadata@<sha>   # reads metadata; runs no PR code
        id: meta
      - if: >                    # conservative scope: patch-only, dev/docs/actions
          steps.meta.outputs.update-type == 'version-update:semver-patch' &&
          steps.meta.outputs.dependency-type == 'direct:development'
        run: gh pr merge --auto --squash "$PR_URL"   # waits for green required checks
# security PRs are never auto-merged; torch/cuda/isaac/numpy excluded by allowlist
```

Phase 3

Gated GPU end-to-end — the capstone

The only tier that proves the GPU runtime — when funded

The GPU half is never proven

- Safe-to-merge cannot be asserted — nothing exercises the GPU runtime
- Only real hardware catches CUDA / driver / Isaac Vulkan / MIG breaks
- #958's device-ABI half passes every cheap tier, then breaks on the GPU
- The blocker is funding: a dedicated, budget-capped GPU subscription

What others do — NeMo gates the GPU run

Queue/human approval + fork mirroring — no pull_request_target

NVIDIA-NeMo/NeMo · cicd-main.yml

```
# NeMo gates the expensive GPU run behind a human/queue approval
cicd-wait-in-queue:
  environment: test          # PAUSES until a reviewer/queue-bot approves
  # ...all GPU jobs `needs: [cicd-wait-in-queue]`
# fork PRs are mirrored to `pull-request/NNN` branches by a bot,
# so untrusted code never runs via pull_request_target ("pwn request")
runner: nemo-ci-aws-gpu-x2   # self-hosted GPU; --gpus all
```

Recommendation — the gated GPU job

Two jobs: PR code renders a spec with no secrets; trusted base code submits after approval

Proposed: gated GPU e2e

```
# Two jobs: untrusted PR code never runs on the token-bearing runner.
# Job A — on: pull_request (fork-safe: read-only token, NO secrets, no cloud login)
jobs:
  render-spec:
    steps:
      - run: ./scripts/render-osmo-spec.sh > job.json   # build a constrained spec
      - uses: actions/upload-artifact@<sha>             # hand off job.json only
# Job B — on: pull_request_review (approved) → runs TRUSTED base-branch workflow
  submit-gpu:
    if: github.event.review.state == 'approved'
    environment: e2e-approval        # required-reviewers gate releases OIDC
    steps:
      - uses: actions/checkout@<sha>            # BASE ref, not the PR head
      - uses: actions/download-artifact@<sha>   # the job.json from Job A
      - run: ./scripts/validate-spec.sh job.json   # allowlist-check before submit
      - uses: azure/login@v2                    # OIDC — minted only after approval
      - run: ./training/rl/scripts/submit-osmo.sh job.json  # PR code runs in the pool
```

What funding buys

Illustrative figures — a budget-capped subscription, not a blank cheque (confidence: low on exact $)

Per gated run	~$3–8 GPU-hour × ~0.3–1.0 hr; GPU node scales from zero between runs
Cluster standing	AKS control plane + system node pool run 24/7 — can't scale to zero ≈ $150–250/mo dedicated, or ~$0 marginal if folded onto a cluster we already run
Frequency	only on maintainer approval — ~5–15 runs/week, not per-PR
Caps	60-min timeout/run · concurrency 1 · monthly budget cap
Scenarios (all-in)	low ≈ $200/mo · expected ≈ $300/mo · spike ≈ $550/mo — GPU compute + standing cluster
Buys	the only proof of CUDA / Vulkan / MIG / training-loop safety
Unfunded risk	GPU-only regressions (e.g. #958 device half) keep merging blind

The decision

Roadmap and the ask

What ships now, what waits on funding, and what we need approved today

Roadmap — ship now vs funded

Phase 0 now · hours	**Risk-aware intake**	group by update-type · cooldown · security stays a fast lane	**$0**
Phase 1 now · days	**GPU-free smoke gate**	1a Tier 0 every PR · 1b Tier 1 real-image, path-gated · fail-safe check	**$0**
Phase 2 now · days	**Safe automation**	patch-only auto-merge on green · scoped manual review for the rest	**$0**
Phase 3 when funded	**Gated GPU e2e**	approval-gated, two-job OIDC submit-and-poll to scale-from-zero pool	**GPU $**
Spike parallel	**Renovate evaluation**	github-action, time-boxed; switch only if PR volume drops	**$0**

Decision requested today

- Approve the Phase 0 `dependabot.yml` change (grouping + cooldown)
- Approve the Phase 1a Tier-0 required check on every PR
- Approve a Phase 1b Tier-1 real-image spike with a runtime cap
- Approve a Phase 2 patch-only auto-merge pilot (dev/docs/actions)
- Approve a time-boxed Renovate spike via github-action
- Defer Phase 3 pending a GPU budget number — design is settled
- Out of band, now: fix the live torch 2.10 / 2.11 desync

Discussion

Questions

Appendix follows: primers, prototype detail, costs, and alternatives

Appendix A

Primers & glossary

The tool concepts referenced throughout — pull up on demand

Primer — Dependabot

Dependabot has TWO independent update streams: scheduled VERSION updates (configured in `dependabot.yml`) and always-on, advisory-driven SECURITY updates (automatic — no config needed). Security PRs are never batched behind version updates. `groups:` batches, `ignore:` pins, `open-pull-requests-limit` caps noise, `cooldown` adds a stability window. It regenerates lockfiles natively.

.github/dependabot.yml

```
# version updates (scheduled) — one stream
- package-ecosystem: uv
  directory: /training/il/lerobot
  groups:                          # batch into 1 PR
    lerobot-deps: { patterns: ["*"] }
  ignore:                          # deliberate pin
  - dependency-name: torch
    versions: [">=2.11.0"]
  open-pull-requests-limit: 5
# security updates = a SEPARATE, automatic stream
```

Key terms: version vs security, group, ignore, cooldown, PR limit, lockfile

Primer — uv and lockfiles

`pyproject.toml` is the human-authored manifest (PEP 621). `uv.lock` is the resolver's exact output — pinned versions, hashes, platform markers. A PR can edit the manifest but forget the lock, so this repo enforces `uv lock --check` as a CI gate.

pyproject.toml + uv.lock

```
# pyproject.toml — human-authored (PEP 621)
[project]
requires-python = ">=3.12"
dependencies = ["lerobot==0.5.1"]

# uv.lock — resolver output: exact
#   versions + hashes + platform markers
# CI gate: `uv lock --check`  (no drift)
```

Key terms: pyproject.toml, PEP 621, uv, uv.lock, resolver, lock drift

Primer — security advisories (GHSA)

A GHSA records a vulnerable package, its affected and patched versions, and a CVSS severity, usually linked to a CVE. When a vulnerable dependency is in your graph, Dependabot opens a security PR to the minimum patched version. These are fast-tracked by severity — never batched or held in a cooldown.

```
a GHSA, conceptually

GHSA-xxxx-xxxx-xxxx   (-> a CVE)
package:    urllib3
vulnerable: < 2.7.0
patched:    >= 2.7.0
severity:   high   (CVSS)
=> security PR: bump to min patched
=> never batch / delay these
```

Key terms: GHSA, CVE, CVSS, severity, advisory, patched version

Primer — CI gating tiers

Cheap checks (lint, spell, lock-consistency) run on every PR; expensive checks run only when their area changed or a human releases them. This repo computes path booleans in one `changes` job and aggregates into ONE stable required check. The trap: a naive top-level `paths:` filter can leave a required check skipped — green having tested nothing.

```
.github/workflows/pr-validation.yml

jobs:
  changes:                      # 1 cheap job computes
booleans
    outputs: { training: ... }
  pytest-training:
    needs: changes
    if: needs.changes.outputs.training == 'true'
  pr-validation-summary:        # 1 stable required
check
    if: always()                # aggregates
everything
```

Key terms: cheap vs expensive, path gate, required check, branch protection, aggregator

Primer — running untrusted PR code

On `pull_request`, fork PRs get a read-only token and NO secrets — safe to build untrusted code. On `pull_request_target` the job runs in the base context WITH secrets; checking out the PR head there is the classic 'pwn request'. Gate real cloud access behind an Environment, and use OIDC with a tight federated policy.

Key terms: fork PR, pull_request vs pull_request_target, pwn request, Environment, OIDC

```
the safe pattern

on: pull_request          # fork PR: read-only token,
NO secrets  (safe)
# on: pull_request_target # base context, HAS secrets
-> "pwn request"
jobs:
  gpu:
    environment: gpu-approved   # gate runs BEFORE any
token is minted
    permissions: { contents: read, id-token: write }
# OIDC, no stored secret
    steps: [ { uses: azure/login@<sha> } ]
# id-token:write is safe only if the cloud federated
policy pins
#   sub / aud / repo / environment — fork refs must
NOT match it
```

Glossary

Dependabot — GitHub's native dependency-update bot

Renovate — third-party update bot; cross-ecosystem config

GHSA — GitHub Security Advisory for a vulnerability

CVSS — standard 0-10 vulnerability severity score

lockfile — pinned exact resolved dependency graph

uv — fast Python resolver; reads pyproject, writes uv.lock

PEP 621 — standard [project] metadata in pyproject.toml

semver — patch / minor / major compatibility levels

required check — status check branch protection insists on

OIDC — short-lived federated cloud login; no stored secret

Environment — deploy target w/ required reviewers + secrets

pull_request_target — base-context trigger WITH secrets (risky)

MIG — one datacentre GPU sliced into isolated instances

Vulkan — graphics API Isaac Sim needs to render

ABI — compiled binary contract; mismatch crashes at runtime

submit-and-poll — CI submits a job to a GPU pool, then waits

smoke test — tiny fast check that the critical path works

Appendix B

Phase mechanics

How each recommendation works in detail — ordered Phase 0 → Phase 1

Renovate — adoption across Microsoft OSS

Phase 0 · We checked directly — it shapes the approval friction

```
evidence: Sourcegraph + file fetches

 Microsoft OSS, by the evidence (Sourcegraph + file fetches):
   Dependabot : the de-facto standard — vscode, TypeScript,
                dotnet/runtime, fluentui, semantic-kernel, …
   Renovate   : ~19 microsoft-org repos (~1.5-3%)
                mostly the VS-team cluster using a shared preset
                github>microsoft/vs-renovate-presets
   Azure: 1   dotnet: ~9   github org: 0
   OSPO page: GitHub-native dep features only (no Renovate mention)
 Escape hatch: `renovatebot/github-action` (self-hosted)
               -> NO Mend App approval needed
```

Renovate — the scoped spike config

Phase 0 · Run via renovatebot/github-action; decide on PR-volume merits

Proposed (spike): renovate.json

```
// run via renovatebot/github-action — no Mend App to approve
{
  "extends": ["config:best-practices"],
  "minimumReleaseAge": "3 days",
  "packageRules": [
    { "matchManagers": ["npm","pep621","terraform","gomod"],
      "groupName": "all-minor-patch",
      "matchUpdateTypes": ["minor","patch"], "automerge": true },
    { "matchPackageNames": ["torch"], "allowedVersions": "<2.11.0" }
  ]
}
```

These are production contracts, not toy configs

Phase 1 · e.g. the AzureML job contract for an Isaac Lab GPU training container

training/rl/workflows/azureml/train.yaml

```
environment_variables:
  PYTHON: /workspace/isaaclab/isaaclab.sh -p
  ACCEPT_EULA: "Y"
  PRIVACY_CONSENT: "Y"
  HYDRA_FULL_ERROR: "1"
  TRAINING_CHECKPOINT_OUTPUT: ${{outputs.checkpoints}}
  # Do NOT add ${{outputs.X}} env vars here — AzureML does not
  # substitute template expressions in environment_variables
  # (only inside command:). Training reads $AZURE_ML_OUTPUT_<NAME>.
compute: azureml:gpu-cluster
environment: azureml:isaaclab-training-env:latest
```

Tier 1 — one image per job, disk-gated

Phase 1 · Disk is the binding constraint, not capability

smoke-environments.yml

```
# one image per job (can't co-resident 18+18+9 GB) · path-gated
matrix:
  - {dir: training/rl,         image: isaac-lab:2.3.2}          # on training/rl/**
  - {dir: training/il/lerobot, image: pytorch:2.11.0-cuda12.8}  # on training/il/**
  - {dir: evaluation,          image: <build Dockerfile.lerobot-eval>}
steps:
  - uses: jlumbroso/free-disk-space@<sha>   # Isaac ~18-22 GB unpacked
  - run: docker system prune -af            # between matrix legs
```

A small refactor widens the Isaac smoke

Phase 1 · Move AppLauncher into main() — like skrl already does (with real acceptance criteria)

training/rl/scripts/rsl_rl/train.py

```
# training/rl/scripts/rsl_rl/train.py — a small refactor widens CPU smoke
# BEFORE (module top level): even `--help` needs a GPU
app_launcher = AppLauncher(args_cli)        # <- runs at import

# AFTER: guard it in main() (like skrl_training.py already does)
def main():
    app_launcher = AppLauncher(args_cli)    # GPU only when actually run
# Acceptance: imports on CPU · --help exits pre-AppLauncher · argv preserved
#   · GPU launch + Hydra/MLflow side-effects unchanged · covered by a test
```

Smoke operating cost and the fail-safe gate

Phase 1 · Who owns it, what it costs to run, and why a skip can't pass as green

Proposed: pr-smoke-summary

```
# Fail-safe path gate: the required check ALWAYS runs and is never 'skipped'.
jobs:
  changes:                    # detect which areas moved (this job is tested)
    outputs: { rl: ..., il: ..., lock: ... }
  smoke-rl:
    needs: changes
    if: needs.changes.outputs.rl == 'true' || needs.changes.outputs.lock == 'true'
  pr-smoke-summary:           # the ONE required check for branch protection
    needs: [changes, smoke-rl, smoke-il]
    if: always()              # never 'skipped' -> a skip can't masquerade as green
    run: |                    # any lockfile change forces the heavy legs;
      [ "$rl" = success ] || [ "$rl_skipped_ok" ] || exit 1   # a wrongly-skipped leg FAILS
```

Appendix C

The case — economics, objections, alternatives

Why it pays, the pushback answered, and the paths we rejected

Economics — the toil being priced out

Order-of-magnitude, to compare against the status quo (confidence: moderate)

Volume	~24 dependency PRs/week (~350 opened, ~216 merged all-time)
Reviewer toil	~24 PRs × a few minutes triage + rebase comments each week
Phase 0+2 saves	batching + patch auto-merge removes most trivial PRs from the queue
Status quo cost	~8 runtime incidents over the window; mean time-to-diagnose in hours

Anticipated objections

Won't auto-merge cause incidents?	patch-only, dev/docs/actions, no runtime/GPU pkgs, no security batch, checks green, instant revert
Why not just pin Python everywhere?	necessary but insufficient — pins don't exercise real-image installs, transitive ABI, or CUDA/MIG
Why not replace Dependabot now?	native grouping covers most of it; the Renovate App is niche in MSFT OSS — spike first
Why not GPU on every PR?	cost, plus running fork code on a GPU runner; gate + submit-and-poll instead
Is emulated-amd64 proof representative?	enough for interpreter/marker breaks; final confidence needs a native amd64 runner

Alternatives considered and rejected

- A custom bot replacing Dependabot — reinvents native grouping; high upkeep
- Immediate Renovate via the Mend App — niche in MSFT OSS; approval friction
- pull_request_target + PR-head checkout for fork creds — the classic pwn request
- GPU on every PR / self-hosted runner running PR code — too costly and risky unfunded
- Pin Python everywhere and stop — necessary but doesn't exercise real installs or device ABI

Mandate and method

- Maintainers' ask: "solve dependabot at the root, and gate safe merges"
- Six parallel research threads, each a cited evidence file
- Grounded in the repo's git history, configs, PRs, and primary upstream sources
- Plus an executed CPU prototype inside the real Isaac image (this session)
- Full research: .copilot-tracking/research/2026-06-19/